

Midterm Exam

October 24 (Friday), 2025

Student ID	Name	Score

Table 1: Student Information Table

(Python Basics)

Question 1 (1 point)

What will the following code print ?

```
import copy

original = [[1, 2], [3, 4]]
a = original
original[0][1] = 9

b = copy.deepcopy(a)
a[1][1] = 10

print(original)
print(b)
```

Question 2 (1 point)

What will this code print?

```
for i in range(1, 6):
    if i == 2:
        continue
    elif i == 4:
        break
    print(i)
```

Question 3 (1 point)

What will the following code print ?

```
def f(x, *args, **kwargs):  
    print(args, x)  
  
f(1, 2, 3, 'hello', age=20)
```

Question 4 (2 points)

Write a **lambda expression** that defines `add_lambda`, which acts like the `add` function, so that the second print statement produces the same result as the first one.

```
def add(x, y):  
    return x + y  
  
print(add(3, 5))          # 8  
print(add_lambda(3, 5))  # 8 Define add_lambda using a lambda expression
```

Question 5 (2 points)

Design your own class called **Adder** and write it on your answer sheet. When an instance of this class is called as a function that takes two numbers, `x` and `y`, the result of `x + y` should be returned as follows:

```
my_add = Adder()  # Create an instance of Adder  
result = my_add(2, 3)  # Call it like a function  
  
print(result)  # 5
```

Question 6 (1 point)

What will this code print?

```
class Dog():
    def __init__(self):
        print('Dog')

    def bark(self):
        print('Mung')

class Bulldog(Dog):
    def __init__(self):
        print('Bulldog')

    def bark(self):
        print('Kaw')

myDog= Bulldog()
myDog.bark()
```

Question 7 (1 point)

What will the following code print ?

```
def my_func(start):
    while start > 0:
        yield start
        start -= 2

for num in my_func(6):
    print(num)
```

Question 8 (2 points)

What will this code print?

```
num = [1, 3, 5]

arr = list(map(lambda x: x-1, num))
print(arr)
```

Question 9 (3 points)

In Python, decorators are functions that modify the behavior of other functions. Below is an example decorator. What will this code print?

```
def require_admin(func):
    def wrapper(user):
        if user != "admin":
            print("Access denied!")
        else:
            func(user)
    return wrapper

@require_admin
def delete_user(user):
    print(f"User deleted by {user}")

delete_user("guest")
delete_user("admin")
```

Question 10 (2 points)

Recall that lists in Python hold everything in memory, while generators in Python generate values on demand, making them more memory-efficient. What will this code print?

```
L = [x**2 for x in range(100)] # List comprehension
G = (x**2 for x in range(100)) # Generator expression

print(L[:5])

print(next(G))
print(next(G))
```

(Useful Libraries: NumPy, PyTorch)

Question 11 (1 point)

What will this code print?

```
import numpy as np

c = np.arange(6)
d = c.reshape(-1, 3)
print(d)
```

Question 12 (1 point)

Suppose you have a training dataset with 2,000 samples and you train a model using a batch size of 500 for 5 epochs. How many iterations occur in total during training?

Question 13 (1 point)

What will this code print?

```
import numpy as np

A = np.array([[1, 2],
               [3, 4]])
b = np.array([1, 0])

print(b @ A)
print(A @ b)
```

Question 14 (2 points)

What will this code print?

```
import numpy as np

A = np.array([[1, 2], [3, 4], [5, 6]])
B = np.array([[[1, 2, 3], [4, 5, 6]],
              [[7, 8, 9], [10, 11, 12]]])

print(A.shape)
print(B.shape)
```

Question 15 (3 points)

What will this code print?

```
import torch
import torch.nn as nn

# Hint: Create a linear layer:  $y = xW + b$ 
layer = nn.Linear(in_features=4, out_features=2)

x = torch.randn(5, 4)
y = layer(x)

print(y.shape)
```

Question 16 (7 points)

In the following code, `print(y.shape)` produces `torch.Size([10, 5, 16, 16])`. Please explain how the last two values, 16 and 16 in `torch.Size([10, 5, 16, 16])` are obtained.

```
import torch
import torch.nn as nn

# 2D Conv
x = torch.randn(10, 3, 32, 32) # batch=10, 3 channels (RGB), image 32 X 32
conv2d = nn.Conv2d(in_channels=3, out_channels=5, kernel_size=4,
                   stride=2, padding=1)
y = conv2d(x)
print(y.shape) # torch.Size([10, 5, 16, 16]) why ?
```

Question 17 (7 points)

What will this code print? (Hint: see the result of code in Question 14) You should provide some intermediate equations to get the result of `print(y.shape)`.

```
import torch
import torch.nn as nn

x = torch.randn(20, 5, 30, 30) # batch=20, 5 channels, feature map 30 X 30
conv2d = nn.Conv2d(in_channels=5, out_channels=10, kernel_size=6,
                   stride=2, padding=2)
y = conv2d(x)
print(y.shape) # torch.Size([?, ?, ?, ?])
```

Question 18 (7 points)

- (a) (2 points) What will this code print?
- (b) (5 points) Please provide at least two advantages of using Max Pooling in the convolutional neural network (CNN).

```
import torch
import torch.nn as nn

# 2D Max Pooling
x = torch.randn(10, 2, 8, 4) # [batch, channel, height, width]
pool2d = nn.MaxPool2d(kernel_size=2, stride=2)
print(pool2d(x).shape) # torch.Size([?, ?, ?, ?])
```

Question 19 (3 points)

What will this code print?

```
import torch
import torch.nn as nn

# Create a simple 4 X 4 input tensor (batch size = 1, one channel)
x = torch.tensor([[
    [2, 3, 2, 0],
    [5, -2, 2, 8],
    [-1, -6, 7, 3],
    [-4, -5, 4, 2]]])

# Define pooling layers
max_pool= nn.MaxPool2d(kernel_size=2, stride=2)

# Apply pooling
max_out= max_pool(x)
print(max_out) # Hint: tensor([[[[?, ?], [?, ?]]]])
```


(Neural Network Basics)**Question 20** (12 points)

`tolist()` is a PyTorch Tensor method that converts a tensor into a regular Python list. It's useful when you want to print or manipulate tensor data in a more human-readable format.

```
import torch

data = torch.tensor([[1, 2, 3], [4, 5, 6]])
print(data)
print(data.tolist())    #  [[1, 2, 3], [4, 5, 6]]
```

The outputs of the above code is as follows:

```
tensor([[1, 2, 3],
        [4, 5, 6]])
[[1, 2, 3], [4, 5, 6]]
```

The following code shows an example of PyTorch Dataset and DataLoader. Recall that Dataset defines what your data is and how to access it while DataLoader handles batching, shuffling, and parallel loading. What will the following code print?

```
import torch
from torch.utils.data import Dataset, DataLoader

# 1. Define a simple custom dataset
class MyDataset(Dataset):
    def __init__(self):
        self.data = [x for x in range(5)]
        self.labels = [x * 2 for x in range(5)]

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        return self.data[idx], self.labels[idx]

# 2. Create dataset and dataloader
dataset = MyDataset()
loader = DataLoader(dataset, batch_size=2, shuffle=False)

# 3. Iterate through the dataloader
for data, label in loader:
    print(f"Data: {data.tolist()}, Label: {label.tolist()}")
```

Question 21 (5 points)

Please remove all Dropout layers from the following code, and insert Batch Normalization (`nn.BatchNorm1d`) after each fully connected layer. Please modify the code below directly.

```

import torch
import torch.nn as nn
import torch.optim as optim

# 1. Dummy dataset
x = torch.randn(100, 1)
y = 2 * x + 1

# 2. Simple model
class SimpleNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(1, 16)
        self.dropout = nn.Dropout(p=0.5)
        self.fc2 = nn.Linear(16, 1)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.dropout(x)
        return self.fc2(x)

model = SimpleNet()
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

# 3. Training loop
for epoch in range(5):
    model.train() # enable dropout
    optimizer.zero_grad()
    y_pred = model(x)
    loss = criterion(y_pred, y)
    loss.backward()
    optimizer.step()
    print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")

# 4. Evaluation (no dropout)
model.eval()

with torch.no_grad():
    test_x = torch.tensor([[1.0]])
    print("Prediction (eval mode):", model(test_x))

```

Question 22 (8 points)

Please explain dropout, batch normalization, and layer normalization.

(Convolutional Neural Network (CNN))

Question 23 (7 points)

Dilation controls the spacing between elements (weights) inside the convolution kernel. Normally, in a standard convolution (dilation=1), the filter slides over consecutive elements of the input. Let us consider 1D CNN and its Kernel size is 3. What is the size of receptive field when dilation = 20 ? (Hint: see Figure 1)

Example 1D Kernel size : 3

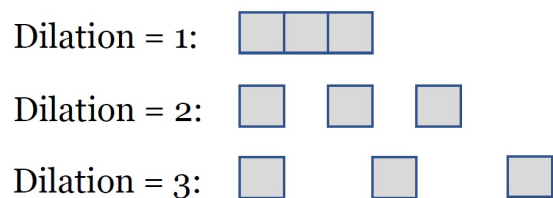


Figure 1: Dilation and Receptive Field in 1D CNN

Question 24 (20 points)

The following code contains three bugs (i.e., three logical errors). Please correct them.

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()

        self.conv1 = nn.Conv2d(in_channels=3, out_channels=16,
                                kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 32, kernel_size=3, padding=1)

        self.pool= nn.MaxPool2d(2, 2)

        self.fc1 = nn.Linear(32 * 8 * 8, 128) # CIFAR10 images are 32 X 32
        self.fc2 = nn.Linear(128 * 2, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 32 * 8 * 8) # Flatten
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        return x

model = SimpleCNN()
print(model)

```